6

Динамический SQL

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Рассматриваемые вопросы

- **Описание потока выполнения команд SQL.**
- **Динамическое построение и выполнение команд SQL с использованием Native Dynamic SQL (с помощью команды `EXECUTE IMMEDIATE`).**
- **Сравнение возможностей, предоставляемых Native Dynamic SQL и пакетом `DBMS_SQL`.**

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Цели урока

На этом уроке вы научитесь динамически строить и выполнять команды SQL. Для этого в PL/SQL используются команды Native Dynamic SQL. Будет проведено сравнение Native Dynamic SQL с пакетом `DBMS_SQL`, предоставляющим похожие возможности.

Поток выполнения кода SQL

- **Шаги обработки команды SQL:**
  - **Разбор (Parse)**
  - **Связывание (Bind)**
  - **Выполнение (Execute)**
  - **Выборка (Fetch)**
- **При выполнении команд определенного вида могут не выполняться некоторые шаги, например, выборка (fetch) производится только для запросов.**

Примечание. Для встроенных команд SQL (`SELECT`, DML, `COMMIT` и `ROLLBACK`) стадии разбора и связывания выполняются во время компиляции. Для динамических команд SQL все шаги производятся во время выполнения.

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Шаги обработки команды SQL

Обработка всех команд SQL проходит в несколько шагов, некоторые из которых могут быть пропущены. Основные шаги следующие.

- **Разбор (Parse).** Каждая команда SQL должна быть разобрана. *Синтаксический разбор* (*parsing*) включает проверку синтаксиса команды, корректности всех ссылок на объекты, а также прав использования этих объектов.
- **Связывание (Bind).** После шага разбора серверу Oracle могут понадобиться значения, извлекаемые из связанных переменных команды или передаваемые в них. Процесс получения этих значений называется связыванием (*binding*). Этот шаг пропускается, когда команда не содержит связанные переменные.
- **Выполнение (Execute).** На этом шаге Oracle имеет всю необходимую информацию и ресурсы, и команда выполняется. Для команд, не относящихся к запросам, – это последний шаг.
- **Выборка (Fetch).** На этом шаге строки выбираются в определенном порядке (если это задано в запросе). Каждая последующая операция *fetch* выбирает следующую строку из результирующего набора. Эта операция выполняется до тех пор, пока не будет выбрана последняя строка. Шаг выборки необходим только для запросов.

Динамический SQL

Используйте динамический SQL для создания команды SQL, структура которой может изменяться во время выполнения. Динамический SQL:

- **Позволяет сконструировать и хранить код в приложении в виде символьной строки.**
- **Позволяет выполнять команду SQL с различными столбцами или условиями, содержащую и не содержащую метки-заполнители (связанные переменные).**
- **Позволяет написать и выполнить в коде PL/SQL команды DDL, DCL и команды управления сеансом**
- **Выполняется с помощью команд Native Dynamic SQL или пакета `DBMS_SQL`.**

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Динамический SQL

Перечень встроенных (embedded) команд SQL, доступных в PL/SQL, ограничен командами `SELECT`, `INSERT`, `UPDATE`, `DELETE`, `COMMIT` и `ROLLBACK`. Такие команды разбираются во время компиляции, то есть их структура неизменна. Возможности динамического SQL могут потребоваться, когда:

- структура команды SQL должна быть изменена во время выполнения;
- необходимо использовать в PL/SQL команды DDL и другие функциональные возможности SQL.

Для решения таких задач в PL/SQL необходимо динамически конструировать команды SQL в символьных строках и выполнять их, используя одну из следующих возможностей:

- команды Native Dynamic SQL, выполняемые с помощью `EXECUTE IMMEDIATE`;
- пакет `DBMS_SQL`.

Термин “динамический SQL” отображает процесс использования команд SQL, которые не встроены в исходную программу, а конструируются в строках и исполняются во время выполнения кода PL/SQL. Команды SQL создаются динамически во время выполнения, они могут иметь доступ к переменным PL/SQL и их использовать. Например, динамический SQL необходим для создания процедуры, в которой производятся операции над таблицей, имя которой неизвестно до выполнения кода PL/SQL. Возможности динамического SQL также необходимы для выполнения команд DDL (например, `CREATE TABLE`), DCL (например, `GRANT`), а также команд управления сеансом (`ALTER SESSION`).

Native Dynamic SQL

- **Предоставляет внутреннюю поддержку для динамического SQL непосредственно в языке PL/SQL**
- **Предоставляет возможность выполнения команд SQL, структура которых неизвестна до начала выполнения**
- **Следующие команды PL/SQL поддерживают эту возможность:**
  - **`EXECUTE IMMEDIATE`**
  - **`OPEN-FOR`**
  - **`FETCH`**
  - **`CLOSE`**

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Native Dynamic SQL (Собственный динамический SQL)

В Oracle 8 и более ранних версиях единственный метод реализации динамического SQL в приложении PL/SQL основывался на использовании пакета `DBMS_SQL`. В Oracle 8*i* и последующих версиях среда PL/SQL предоставляет в качестве альтернативы *собственную поддержку динамического SQL* (*Native Dynamic SQL*).

Native Dynamic SQL позволяет выполнять команды SQL, формируемые динамически во время выполнения. Для поддержки возможностей Native Dynamic SQL в PL/SQL были добавлены следующие команды:

- **`EXECUTE IMMEDIATE`** подготавливает команду, выполняет ее, возвращает переменные и затем освобождает ресурсы;
- **`OPEN-FOR`** подготавливает и выполняет команду, используя курсорную переменную;
- **`FETCH`** возвращает результаты открытой команды, используя курсорную переменную;
- **`CLOSE`** закрывает курсор с помощью соответствующей курсорной переменной и освобождает ресурсы.

Связанные переменные можно использовать в динамических параметрах команд `EXECUTE IMMEDIATE` и `OPEN`. Возможности Native Dynamic SQL:

- определение динамической команды SQL;
- связывание экземпляров данных любого SQL-типа, поддерживаемого в PL/SQL;
- обработка связанных переменных в режиме `IN`, `IN OUT` и `OUT`, соответствие которых устанавливается позиционно, а не по имени.

Использование команды EXECUTE IMMEDIATE

Используйте EXECUTE IMMEDIATE для внутреннего выполнения динамической команды SQL или анонимного блока PL/SQL:

```
EXECUTE IMMEDIATE динамическая_строка
 [INTO {определенная_переменная
     [, определенная_переменная] ... | запись}]
 [USING [IN|OUT|IN OUT] связанный_аргумент
     [, [IN|OUT|IN OUT] связанный_аргумент] ... ];
```

- **Предложение INTO используется для однострочных запросов, и в нем задаются переменные или записи, в которые заносятся выбираемые значения столбцов.**
- **Предложение USING используется для обработки всех связанных аргументов. По умолчанию, если режим не задан, для параметра действует режим IN.**

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Использование команды EXECUTE IMMEDIATE

Команда EXECUTE IMMEDIATE используется для выполнения команд SQL и анонимных блоков PL/SQL. Синтаксические элементы этой команды:

- динамическая_строка отображает динамическую команду SQL (без терминатора) или блок PL/SQL (без терминатора).
- определенная_переменная – переменная PL/SQL, в которой сохраняется выбираемое значение столбца.
- запись – определяемая пользователем или с помощью атрибута %ROWTYPE запись, используемая для хранения выбранной строки.
- связанный_аргумент – выражение, значение которого передается внутренне исполняемой динамической команде SQL или блоку PL/SQL.
- INTO–предложение содержит переменные или записи, в которые помещаются выбираемые значения столбцов. Оно используется только для однострочных запросов. В предложении INTO для каждого значения, выбираемого запросом, должна быть совместимая по типу переменная или поле.
- USING–предложение содержит все связанные аргументы. По умолчанию их режим – IN.

В качестве связанных аргументов можно использовать числовые, символьные и строковые литералы, но не логические (TRUE, FALSE или NULL).

Примечание. Используйте команды OPEN-FOR, FETCH и CLOSE для многострочного запроса. На слайде приведен неполный синтаксис, так как в нем не отражены *операции массовой обработки* (*bulk-processing operations*), не рассматриваемые в этом курсе.

Использование Native Dynamic SQL для выполнения команды DDL

- **Процедура создания таблицы:**

```
CREATE PROCEDURE create_table(
  table_name VARCHAR2, col_specs  VARCHAR2) IS
BEGIN
  EXECUTE IMMEDIATE 'CREATE TABLE '||table_name||
                    ' (' || col_specs || ')';
END;
/
```

- **Пример вызова процедуры:**

```
BEGIN
  create_table('EMPLOYEE_NAMES',
   'id NUMBER(4) PRIMARY KEY, name VARCHAR2(40)');
END;
/
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Использование Native Dynamic SQL для выполнения команды DDL

Пример на слайде показывает создание процедуры `create_table`, которая принимает имя таблицы и определения (спецификации) столбцов в качестве параметров.

Вызов этой процедуры, приведенной на слайде, создает таблицу `EMPLOYEE_NAMES` с двумя столбцами:

- `ID` с типом данных `NUMBER`, используемым в качестве главного ключа;
- `name` длиной до 40 символов для фамилии сотрудника.

Любая команда DDL может быть выполнена с использованием синтаксиса, показанного на слайде, независимо от того, конструируется ли команда динамически или задается в виде строкового литерала. Вы можете создать и выполнить команду, хранимую в строковой переменной PL/SQL, как в следующем примере :

```
CREATE PROCEDURE add_col(table_name VARCHAR2,
                         col_spec VARCHAR2) IS
  stmt VARCHAR2(100) := 'ALTER TABLE ' || table_name ||
                        ' ADD '||col_spec;
BEGIN
  EXECUTE IMMEDIATE stmt;
END;
/
```

Для добавления нового столбца в таблицу введите:

```
EXECUTE add_col('employee_names', 'salary number(8,2)')
```

Использование Native Dynamic SQL для выполнения команды DML

- **Удаление строк из таблицы:**

```
CREATE FUNCTION del_rows(table_name VARCHAR2)
RETURN NUMBER IS
BEGIN
  EXECUTE IMMEDIATE 'DELETE FROM '||table_name;
  RETURN SQL%ROWCOUNT;
END;
```

```
BEGIN DBMS_OUTPUT.PUT_LINE(
  del_rows('EMPLOYEE_NAMES')|| ' rows deleted.');
END;
```

- **Вставка строки в таблицу с двумя столбцами :**

```
CREATE PROCEDURE add_row(table_name VARCHAR2,
   id NUMBER, name VARCHAR2) IS
BEGIN
  EXECUTE IMMEDIATE 'INSERT INTO '||table_name||
        ' VALUES (:1, :2)' USING id, name;
END;
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Использование Native Dynamic SQL для выполнения команды DML

Примеры на слайде демонстрируют следующее:

- Функция `del_rows` удаляет строки из заданной таблицы и возвращает количество удаленных строк, используя для этого атрибут `%ROWCOUNT` неявного курсора `SQL`. Выполнение функции показано ниже примера создания функции.
- Процедура `add_row` показывает, как можно предоставить вводимые значения для динамической команды SQL, используя для этого предложение `USING`. Важен порядок, в котором в предложении `USING` указываются имена переменных (`id` и `name`), так как они должны соответствовать позиционно именам связанных переменных `:1` и `:2`. В результате метка-заполнитель `:1` получает значение переменной PL/SQL с именем `id`, а `:2` – `name`. Для *меток-заполнителей/связанных переменных* (*placeholder/bind variable*) задаются алфавитно-цифровые имена и перед ними должно стоять двоеточие.

Примечание. Команда `EXECUTE IMMEDIATE` подготавливает (разбирает) и немедленно выполняет динамическую команду SQL. Динамические команды SQL всегда разбираются (parse).

В обоих примерах не выполняется операция `COMMIT`. Поэтому выполняемые действия можно отменить, используя команду `ROLLBACK`.

Использование Native Dynamic SQL для выполнения однострочного запроса

Пример однострочного запроса:

```
CREATE FUNCTION get_emp(emp_id NUMBER)
RETURN employees%ROWTYPE IS
  stmt VARCHAR2(200);
  emprec employees%ROWTYPE;
BEGIN
  stmt := 'SELECT * FROM employees ' ||
          'WHERE employee_id = :id';
  EXECUTE IMMEDIATE stmt INTO emprec USING emp_id;
  RETURN emprec;
END;
/
```

```
DECLARE
  emprec employees%ROWTYPE := get_emp(100);
BEGIN
  DBMS_OUTPUT.PUT_LINE('Emp: '||emprec.last_name);
END;
/
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Использование Native Dynamic SQL для выполнения однострочного запроса

Функция `get_emp` демонстрирует однострочный запрос, по которому строка из таблицы `EMPLOYEES` выбирается и помещается в переменную, заданную в предложении `INTO`. Эта функция также показывает, как можно предоставить входные значения для предложения `WHERE`.

Для выполнения функции `get_emp` используется анонимный блок, в котором результат, возвращаемый функцией, помещается в локальную переменную, отображающую строку таблицы `EMPLOYEES`.

В зависимости от значений предоставляемых входных параметров пример можно доработать так, чтобы были возможны другие варианты предложения `WHERE`. В результате функция станет более пригодной для обработки динамического SQL.

Использование Native Dynamic SQL для выполнения многострочного запроса

Обработка производится с помощью команд OPEN-FOR, FETCH и CLOSE.

```
CREATE PROCEDURE list_employees(deptid NUMBER) IS
  TYPE emp_refcsr IS REF CURSOR;
  emp_cv  emp_refcsr;
  emprec  employees%ROWTYPE;
  stmt varchar2(200) := 'SELECT * FROM employees';
BEGIN
  IF deptid IS NULL THEN OPEN emp_cv FOR stmt;
  ELSE
    stmt := stmt || ' WHERE department_id = :id';
    OPEN emp_cv FOR stmt USING deptid;
  END IF;
  LOOP
    FETCH emp_cv INTO emprec;
    EXIT WHEN emp_cv%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE(emprec.department_id||
                         ' ' ||emprec.last_name);
  END LOOP;
  CLOSE emp_cv;
END;
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Использование Native Dynamic SQL для выполнения многострочного запроса

Пример на слайде показывает, как выполнить многострочный запрос. Для этого в программе выполняются следующие шаги:

- Объявление типа `REF CURSOR` .
- Объявление курсорной переменной с использованием имени объявленного типа `REF CURSOR` .
- Выполнение команды `OPEN-FOR`, в которой используется курсорная переменная.
- Пока все строки не будут обработаны, выполняется команда `FETCH`, в которой указана курсорная переменная.
- Выполнение команды `CLOSE`, использующей курсорную переменную.

Процесс такой же, как и при использовании статически определяемого курсора. Однако синтаксис команды `OPEN-FOR` позволяет использовать строковый литерал или переменную, задающую команду `SELECT`, которую можно сформировать динамически.

Примечание. Следующая страница кратко знакомит с типом `REF CURSOR` и курсорными переменными. В альтернативном варианте выполнения многострочного запроса используется синтаксис `BULK COLLECT`, поддерживаемый командами Native Dynamic SQL (эта тема не рассматривается в этом курсе).

Объявление курсорных переменных

- **Использование `REF CURSOR` для определение курсорного типа:**

```
CREATE PROCEDURE process_data IS
  TYPE ref_ctype IS REF CURSOR; -- weak ref cursor
  TYPE emp_ref_ctype IS REF CURSOR -- strong
     RETURN employees%ROWTYPE;
  :
```

- **Объявление курсорной переменной с использованием курсорного типа :**

```
  :
  dept_csrvar ref_ctype;
  emp_csrvar  emp_ref_ctype;
BEGIN
  OPEN emp_csrvar FOR SELECT * FROM employees;
  OPEN dept_csrvar FOR SELECT * from departments;
  -- Then use as normal cursors
END;
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Объявление курсорных переменных

Курсорная переменная (*cursor variable*) – это идентификатор PL/SQL, при объявлении которого использовалось имя объявленного до этого типа `REF CURSOR`. Создание курсорной переменной включает два шага:

- объявление имени типа `REF CURSOR`;
- объявление переменной PL/SQL с типом, задаваемым именем объявленного до этого типа `REF CURSOR`.

В примерах на слайде создаются два *типа ссылочных курсоров* (*reference cursor types*):

- `ref_ctype` – ссылочный курсор, являющийся *слабым* (*weak*) ссылочным курсором. Такой курсор может быть связан с любым запросом.
- `emp_ref_ctype` – *строгий* (*strong*) ссылочный курсор, который может быть связан с совместимым по типу запросом. Такой запрос должен возвращать данные в соответствие с типом, заданным после ключевого слова `RETURN` (например, строки таблицы `EMPLOYEES`).

После объявления курсорной переменной с использованием имени типа ссылочного курсора открывается связанная с запросом курсорная переменная (команда `OPEN-FOR`). Обычная команда `FETCH`, атрибуты курсора и операция `CLOSE`, используемые с явными курсорами, также применимы к курсорным переменным. При сравнении курсорных переменных и явных курсоров можно отметить следующее:

- курсорная переменная может связываться во время выполнения с несколькими запросами;
- явный курсор связывается с одним запросом во время компиляции.

Динамическое выполнение блока PL/SQL

Динамическое выполнение анонимного блока PL/SQL:

```
CREATE FUNCTION annual_sal(emp_id NUMBER)
RETURN NUMBER IS
  plsql varchar2(200) :=
    'DECLARE '||
    ' emprec employees%ROWTYPE; '||
    'BEGIN '||
    ' emprec := get_emp(:empid); ' ||
    ' :res := emprec.salary * 12; ' ||
    'END;';
  result NUMBER;
BEGIN
 EXECUTE IMMEDIATE plsql
         USING IN emp_id, OUT result;
  RETURN result;
END;
/
```

```
EXECUTE DBMS_OUTPUT.PUT_LINE(annual_sal(100))
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Динамическое выполнение блока PL/SQL

Функция `annual_sal` динамически конструирует анонимный блок PL/SQL. Этот блок содержит переменные для:

- входного значения номера сотрудника (метка-заполнитель `:empid`);
- выходного результата, представляющего собой подсчитанный годовой оклад сотрудника (метка-заполнитель `:res`).

Примечание

В этом примере демонстрируется синтаксис задания связанных переменных в режиме `OUT`. Они используются для получения выходного результата, рассчитанного в блоке PL/SQL, и задаются в предложении `USING` команды `EXECUTE IMMEDIATE`. Выходные переменные процедуры и возвращаемое значение функции могут быть получены одинаковым путем из динамически выполняемого анонимного блока PL/SQL.

Использование Native Dynamic SQL для компиляции кода PL/SQL

Компиляция кода PL/SQL по команде ALTER:

- ALTER PROCEDURE имя COMPILE
- ALTER FUNCTION имя COMPILE
- ALTER PACKAGE имя COMPILE SPECIFICATION
- ALTER PACKAGE имя COMPILE BODY

```
CREATE PROCEDURE compile_plsql(name VARCHAR2,
 plsql_type VARCHAR2, options VARCHAR2 := NULL) IS
  stmt varchar2(200) := 'ALTER '|| plsql_type ||
                        ' '|| name || ' COMPILE';
BEGIN
 IF options IS NOT NULL THEN
   stmt := stmt || ' ' || options;
 END IF;
 EXECUTE IMMEDIATE stmt;
END;
/
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Использование Native Dynamic SQL для компиляции кода PL/SQL

Приведенная в примере процедура compile_plsql может использоваться для компиляции различного кода PL/SQL с использованием команды DDL ALTER. На слайде приведены четыре основные вида команды ALTER для компиляции:

- процедуры;
- функции;
- спецификации пакета;
- тела пакета.

Примечание: если в команде ALTER PACKAGE опушено ключевое слово SPECIFICATION и BODY, тогда компилируются и спецификация, и тело.

Примеры вызова процедуры, приведенной на слайде, для каждого из четырех случаев:

```
EXEC compile_plsql ('list_employees', 'procedure')
EXEC compile_plsql ('get_emp', 'function')
EXEC compile_plsql ('mypack', 'package', 'specification')
EXEC compile_plsql ('mypack', 'package', 'body')
```

Компиляция функции get_emp с включенной опцией DEBUG:

```
EXEC compile_plsql ('get_emp', 'function', 'debug')
```

Использование пакета DBMS_SQL

Пакет DBMS_SQL используется для написания динамического SQL в хранимых процедурах и для синтаксического разбора команд DDL.

Некоторые процедуры и функции пакета:

- **OPEN_CURSOR**
- **PARSE**
- **BIND_VARIABLE**
- **EXECUTE**
- **FETCH_ROWS**
- **CLOSE_CURSOR**

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Использование пакета DBMS_SQL

С помощью пакета DBMS_SQL можно создавать хранимые процедуры и анонимные блоки PL/SQL, в которых используется динамический SQL. Например, выполнять в коде PL/SQL команды DDL (DROP TABLE и т.п.). Операции, предоставляемые этим пакетом, выполняются в среде текущего пользователя, а не пользователя SYS (владельца пакета). Пакет DBMS_SQL предоставляет следующие подпрограммы для выполнения динамического SQL:

- OPEN_CURSOR открывает новый курсор и возвращает идентификационный номер курсора (cursor ID number).
- PARSE разбирает команду SQL, то есть проверяет синтаксис команды и связывает ее с открытым курсором. Команды DDL немедленно выполняются на шаге разбора.
- BIND_VARIABLE связывает предоставляемое значение со связанной переменной, распознаваемой по ее имени в разобранной команде. Этот шаг не требуется, когда команда не содержит связанные переменные.
- EXECUTE выполняет команду SQL и возвращает количество обработанных строк.
- FETCH_ROWS извлекает следующую строку, возвращаемую запросом (используйте цикл для выбора множества строк).
- CLOSE_CURSOR закрывает заданный курсор.

Примечание. Использование процедуры DBMS_SQL для выполнения команд DDL может привести к взаимоблокировке. Наиболее вероятной причиной для этого может быть попытка удаления процедуры, которую вы все еще используете.

Использование пакета DBMS_SQL для выполнения команды DML

Пример удаления строк:

```
CREATE OR REPLACE FUNCTION delete_all_rows
  (table_name VARCHAR2) RETURN NUMBER IS
  csr_id INTEGER;
  rows_del    NUMBER;
BEGIN
  csr_id := DBMS_SQL.OPEN_CURSOR;
  DBMS_SQL.PARSE(csr_id,
    'DELETE FROM '||table_name, DBMS_SQL.NATIVE);
  rows_del := DBMS_SQL.EXECUTE (csr_id);
  DBMS_SQL.CLOSE_CURSOR(csr_id);
  RETURN rows_del;
END;
/
```

```
BEGIN DBMS_OUTPUT.PUT_LINE('Rows Deleted: ' ||
   delete_all_rows('employees'));
END;
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Использование пакета DBMS_SQL для выполнения команды DML

В примере на слайде процедуре delete_all_rows передается имя таблицы. Функция использует динамический SQL для удаления строк из заданной таблицы и возвращает количество строк, удаленных после успешного выполнения команды.

Чтобы динамически обработать команду DML, выполните следующие шаги:

1. Используйте функцию OPEN_CURSOR для создания области памяти, предназначенной для обработки команды SQL.
2. Используйте процедуру PARSE для проверки команды SQL.
3. Используйте функцию EXECUTE, чтобы выполнить команду SQL. Эта функция возвращает количество обработанных строк.
4. Используйте процедуру CLOSE_CURSOR для закрытия курсора.

При выполнении команды DDL производятся похожие шаги; однако шаг 3 необязателен, поскольку команда DDL немедленно исполняется, как только успешно завершается шаг разбора (PARSE). Это происходит, когда команда синтаксически и семантически корректна. При использовании для команды DDL функции EXECUTE никакие действия не производятся, а только возвращается значение 0, отражающее количество обработанных строк, так как команды DDL строки не обрабатывают.

Использование пакета DBMS_SQL для выполнения команды DML с параметрами

```
CREATE PROCEDURE insert_row (table_name VARCHAR2,
 id VARCHAR2, name VARCHAR2, region NUMBER) IS
  csr_id     INTEGER;
  stmt       VARCHAR2(200);
  rows_added NUMBER;
BEGIN
  stmt := 'INSERT INTO '||table_name||
           ' VALUES (:cid, :cname, :rid)';
  csr_id := DBMS_SQL.OPEN_CURSOR;
  DBMS_SQL.PARSE(csr_id, stmt, DBMS_SQL.NATIVE);
  DBMS_SQL.BIND_VARIABLE(csr_id, ':cid', id);
  DBMS_SQL.BIND_VARIABLE(csr_id, ':cname', name);
  DBMS_SQL.BIND_VARIABLE(csr_id, ':rid', region);
  rows_added := DBMS_SQL.EXECUTE(csr_id);
  DBMS_SQL.CLOSE_CURSOR(csr_id);
  DBMS_OUTPUT.PUT_LINE(rows_added||' row added');
END;
/
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Использование пакета DBMS_SQL для команды DML с параметрами

В примере на слайде выполняется операция DML, вставляющая строку в заданную таблицу. Пример демонстрирует дополнительный шаг, требуемый для сопоставления значений со связанными переменными, имеющимися в команде SQL. Выше приведенную процедуру можно, например, вызвать следующим образом:

```
EXECUTE insert_row('countries', 'ZA', 'South Africa', 4)
```

После разбора команды для присвоения значений каждой находящейся в ней связанной переменной требуется выполнить процедуру DBMS_SQL.BIND_VARIABLE. Связывание значений необходимо произвести до выполнения кода. Чтобы динамически обработать команду SELECT, выполните следующие шаги после открытия и перед закрытием курсора:

1. Выполните процедуру DBMS_SQL.DEFINE_COLUMN для каждого выбираемого столбца.
2. Выполните процедуру DBMS_SQL.BIND_VARIABLE для каждой связанной переменной в запросе.
3. Для каждой строки:
   (а). Выполните функцию DBMS_SQL.FETCH_ROWS, чтобы извлечь строку и вернуть количество извлеченных строк. Остановите дальнейшую обработку, если возвращаемое значение равно нулю.
   (б). Выполните процедуру DBMS_SQL.COLUMN_VALUE для занесения значения каждого выбираемого столбца в переменную PL/SQL.

Хотя закодировать эту обработку несложно, такой подход требует больше времени и более подвержен ошибкам по сравнению с методом, использующим Native Dynamic SQL.

Сравнение Native Dynamic SQL с пакетом DBMS_SQL

Native Dynamic SQL:

- **Проще использовать, чем пакет DBMS_SQL**
- **Требует меньшего кодирования по сравнению с применением пакета DBMS_SQL**
- **Улучшает производительность, поскольку интерпретатор PL/SQL предоставляет для него внутреннюю (native) поддержку**
- **Обрабатывает все типы, поддерживаемые статически SQL в PL/SQL, включая типы, определенные пользователем**
- **Позволяет выбирать строки непосредственно в записи PL/SQL**

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Сравнение Native Dynamic SQL с пакетом DBMS_SQL

Native Dynamic SQL предоставляет следующие преимущества по сравнению с пакетом DBMS_SQL.

Простота использования. Native Dynamic SQL можно использовать таким же образом, как используется статический SQL внутри кода PL/SQL. Код обычно более компактный и читабельный по сравнению с кодом, написанным с использованием пакета DBMS_SQL.

Лучшая производительность. В большинстве случаев Native Dynamic SQL выполняется значительно лучше, чем DBMS_SQL, вследствие *внутренней* (*native*) поддержки, предоставляемой интерпретатором PL/SQL. DBMS_SQL использует процедурный API (application programming interface, программный интерфейс приложения) и неэффективен из-за накладных расходов, вызванных большим количеством вызовов процедур и копирования данных.

Поддержка пользовательских типов. Native Dynamic SQL поддерживает все типы, которые поддерживает статический SQL в PL/SQL. Следовательно, Native Dynamic SQL предоставляет поддержку таких пользовательских типов, как объекты, определенные пользователем, коллекции и ссылки (REF). Пакет DBMS_SQL не поддерживает такие пользовательские типы, но предоставляет ограниченную поддержку массивов.

Поддержка извлечения (fetching) и занесения данных в записи. При использовании Native Dynamic SQL можно извлекать (fetch) строки из результирующего набора

запроса и помещать их непосредственно в записи PL/SQL. Пакет `DBMS_SQL` это не поддерживает.